

A Survey of Implicit Neural Representations for Video Compression

Hannes Keunen¹, Maarten Wijnants¹, and Jori Liesenborgs¹

¹Digital Future Lab

August 02, 2025

Abstract

Neural Video Representations (NVRs) have recently been proposed as a novel approach to the video compression problem. NVRs consist of one or multiple small neural networks that are overfitted on one specific video sequence, thereby encoding the video within the weights and biases of the network(s). In contrast to other learned video coding approaches, NVR-based codecs do not rely on large datasets and have a relatively low decoding complexity. Many works have focused on improving the compression performance of NVR-based codecs by enhancing the overall codec design, devising more performant and parameter-efficient model architectures, and incorporating more advanced model compression schemes such as weight pruning, quantization, and entropy minimization. We provide a systematic overview of representative work in the field and discuss common weaknesses and opportunities for future work, with a focus on practical deployment for video streaming. This paper serves as both an introduction for newcomers and a reference for existing researchers, highlighting the potential of neural video representations as an alternative to traditional codecs in video compression.

A Survey of Implicit Neural Representations for Video Compression

Hannes Keunen^{1*}, Maarten Wijnants¹ and Jori Liesenborgs¹

^{1*}Digital Future Lab, UHasselt - Flanders Make, Diepenbeek, Belgium.

*Corresponding author(s). E-mail(s): hannes.keunen@uhasselt.be;
Contributing authors: maarten.wijnants@uhasselt.be;
jori.liesenborgs@uhasselt.be;

Abstract

Neural Video Representations (NVRs) have recently been proposed as a novel approach to the video compression problem. NVRs consist of one or multiple small neural networks that are overfitted on one specific video sequence, thereby encoding the video within the weights and biases of the network(s). In contrast to other learned video coding approaches, NVR-based codecs do not rely on large datasets and have a relatively low decoding complexity. Many works have focused on improving the compression performance of NVR-based codecs by enhancing the overall codec design, devising more performant and parameter-efficient model architectures, and incorporating more advanced model compression schemes such as weight pruning, quantization, and entropy minimization. We provide a systematic overview of representative work in the field and discuss common weaknesses and opportunities for future work, with a focus on practical deployment for video streaming. This paper serves as both an introduction for newcomers and a reference for existing researchers, highlighting the potential of neural video representations as an alternative to traditional codecs in video compression.

Keywords: Implicit neural representations (INR), learned video compression, model compression, neural video representations (NVR), video streaming

1 Introduction

With video streaming accounting for more than 65% of global internet traffic in 2024¹, the demand for better video codecs is growing rapidly. Traditional video codecs, such as H.266/VVC [1], still rely on hand-crafted features to exploit redundancies within and between frames. Although H.266/VVC achieves approximately 50% bit rate reduction over its predecessor, H.265/HEVC [2], the computational complexity for encoding and decoding also increases dramatically with each new generation of video codecs [3].

Recent advances in machine learning have been pushing towards the use of deep learning techniques for video compression, either by incorporating deep learning models in the traditional video compression pipeline, or by completely replacing the traditional pipeline by one that is trained end-to-end. DVC [4] first achieved this by replacing each component of the traditional video compression pipeline with components that can be optimized jointly. Subsequent work has expanded upon DVC, surpassing HEVC in terms of rate-distortion performance [5, 6]. A more complete overview of deep learning techniques for video compression can be found in [7].

The weakness of end-to-end trained neural compression methods, such as DVC, lies in the fact that they are general-purpose and thus have to perform well on a wide range of videos with heterogeneous characteristics (e.g., live action versus animation, low versus high camera motion, low versus high spatial complexity, and so on). To achieve this, they usually require training on large datasets. This also results in high parameter counts, and thus high decoding complexity and energy usage, making them impractical for deployment, especially on mobile devices.

Implicit neural representations (INRs) represent an alternative approach to video compression. Instead of fitting a large model to an extensive dataset, a much smaller model can be overfitted to memorize a single video. This allows a video to be encoded in the parameters of the network, eliminating the dependence on large datasets and significantly decreasing the decoding complexity. The INR paradigm was first introduced to represent spatial data as a continuous function $f_{\theta} : \mathbb{R}^I \rightarrow \mathbb{R}^O$ [8–10]. This function can be approximated by a neural network parametrized by θ . Several works [11–13] have adapted the INR paradigm to image data by mapping spatial image coordinates to pixel intensities. SIREN [13] first extended this approach to video data by naively generating each pixel individually, indexed by its spatiotemporal coordinate (t, x, y) . NeRV [14] was the first INR that specifically targeted video data, using convolution and upsampling operations to generate entire video frames from a temporal coordinate t . This inspired a new line of research on neural video representations (NVRs), improving NeRV for tasks such as video inpainting, frame interpolation, and video compression. The primary focus of this paper will be on the video compression task.

In the context of NVR, a video $\mathbf{V} \in \mathbb{R}^{T \times W \times H \times C}$, with T frames of size $W \times H$ and C channels per pixel, will be formalized as a set of N functions

$$\mathcal{F}(\mathbf{i}; \Theta) = \{D_n(E_n(\mathbf{i}; \theta_{enc}^n); \theta_{dec}^n)\}_{n=1}^N, \quad (1)$$

$$\Theta = \{\theta_n\}_{n=1}^N = \{\theta_{enc}^n, \theta_{dec}^n\}_{n=1}^N \quad (2)$$

¹<https://www.appligicnetworks.com/phenomena>

where $\mathbf{i}$ represents some coordinate into $\mathbf{V}$, E_n is a function that generates an input embedding $\mathbf{e}_n$ for the region of the video indexed by $\mathbf{i}$, and D_n decodes the corresponding video data from that embedding. As we will explain in Section 2.2, each of the N functions in $\mathcal{F}$ encodes some part of $\mathbf{V}$ in its parameters θ_n . Each function D_n is approximated by a neural network parametrized by θ_{dec}^n . The embedding function E_n may also be in the form of a neural network parameterized by θ_{enc}^n , but can also be a handcrafted embedding. Section 3.4 contains an overview of the different forms of embedding functions. The complete set of parameters is denoted as Θ .

The contributions of this paper are as follows:

- We provide an overview of the existing techniques that can be used for designing approximators for Equation (1) and their respective impact on reconstruction quality and compression performance.
- We discuss how an NVR-based video codec might be employed in the context of video streaming.
- Based on this practical scenario, we identify common challenges related to NVRs and provide opportunities for future work.

Section 2 first provides an overview of existing works by comparing different high-level design considerations for NVR-based video codecs. Section 3 discusses low-level design choices for NVR models. Section 4 discusses several model compression techniques used in NVRs. To identify common weaknesses and opportunities for future work, Section 5 finally discusses how the NVR paradigm might be employed in the practical context of video streaming.

2 High-level Codec Design

We first discuss several high-level design considerations for NVRs. Section 2.1 discusses the effect of *instance size*, or the amount of data generated in each forward pass. Section 2.2 discusses *model scope*, or the amount of data captured by each set of model parameters θ_n . Section 2.3 discusses the different types of *input embedding* that can be used as input to D_n . Finally, Section 2.4 discusses several ways in which temporal information can be exploited. An overview of all the works discussed in this section can be found in Table 1. Table 2 shows PSNR results achieved by various models on the UVG dataset [15], as obtained from the respective papers. Note that this only shows reconstruction quality, as compression performance will be considered in Section 4.

2.1 Instance Size

Instance size is the amount of data generated in a single forward pass of D_n . The format of the input coordinate $\mathbf{i}$ is determined by the size and shape of each instance.

2.1.1 Pixel

Neural representations for images [13, 38] usually generate one pixel at a time, indexed by spatial coordinates (x, y) . SIREN [13] extends this pixel-wise approach to videos, where each individual pixel of $\mathbf{V}$ is indexed by its spatio-temporal coordinate $\mathbf{i} =$

Table 1 Overview of neural video representation methods

Year	Name	Instance	Scope	Input	Temporal context
2021	SIREN [13]	Pixel	Full	/	Implicit
	NeRV [14]	Frame	Full	PE	Implicit
2022	E-NeRV [16]	Frame	Full	PE, Grid	Implicit
	CNeRV [17]	Frame	Full	IE	Implicit
	PS-NeRV [18]	Patch	Full	PE	Implicit
2023	NIRVANA [19]	3D Patch	GoP	PE	Conv3D
	D-NeRV [20]	Frame	Full	PE	ME/MC
	HNeRV [21]	Frame	Full	IE	Implicit
	DNeRV [22]	Frame	Full	IE, TE	Encoder
	FFNeRV [23]	Frame	Full	MTG	ME/MC
	Gomes et al. [24]	Frame	Full	Grid	Implicit
	C3 [25]	3D Patch	Instance	IE	Conv3D
2024	COOL-CHIC video [26]	Frame	Instance	IE	ME/MC
	HiNeRV [27]	Patch	Full	MTG	Implicit
	NeRV++ [28]	Frame	Full	PE	Implicit
	HNeRV-BOOST [29]	Frame	Full	IE	Implicit
	VQ-Nerv [30]	Frame	Full	IE	Implicit
	T-NeRV [31]	Frame	Full	PE, MTG, IE	Implicit
	NVRC [32]	Frame	Full	MTG	Implicit
	TNeRV [33]	Frame	Full	IE, TE	Encoder
	QS-NeRV [34]	Frame	Layer	IE	Implicit
	SNeRV [35]	Frame	Full	IE, TE	Implicit
	HFS-HNeRV [36]	Frame	Full	IE	Implicit
	PNVC [37]	Frame	Instance	IE	ME/MC

(t, x, y) where $t < T$, $x < W$, and $y < H$. Because pixel-wise methods do not explicitly exploit spatial redundancies, this leads to inferior results compared to larger instance sizes.

2.1.2 Frame

NeRV [14] showed that switching to a frame-wise approach with convolutional neural networks substantially reduces encoding and decoding complexity, and improves reconstruction quality. Frames are indexed by a single temporal coordinate $\mathbf{i} = t, t < T$. Many later works [16, 17, 20–24, 26, 28–31, 33–37] followed the same frame-wise approach.

2.1.3 Patch

As a more generalized formulation, $\mathbf{V}$ can be subdivided into patches of size $T_p \times W_p \times H_p$, indexed by patch coordinate $\mathbf{i} = (t, x, y)$ where $t < \frac{T}{T_p}$, $x < \frac{W}{W_p}$, and $y < \frac{H}{H_p}$. As such, the pixel- and frame-wise approaches can both be seen as special cases of the patch-wise approach, where the pixel-wise approach uses a patch size of $1 \times 1 \times 1$ and the frame-wise approach uses a patch size of $1 \times W \times H$. A spatial patch-wise approach ($T_p = 1$) was first introduced by PS-NeRV [18] to better capture high-frequency detail, while still preserving the improvements in computational complexity offered by frame-wise methods over pixel-wise methods. This results in an average

Table 2 PSNR results on the UVG (1080p) dataset [15], trained for 300 epochs with 3M parameters

Year	Model	Beauty	Bosphorus	Honey	Jockey	Ready	Shake	Yacht	Avg.
2021	NeRV [14]	33.14	32.74	37.18	30.99	23.97	33.06	31.11	31.11
2022	E-NeRV [16]	33.29	33.87	38.88	28.73	23.98	34.45	31.51	31.51
	PS-NeRV [18]	32.94	32.32	38.39	30.38	23.61	33.26	26.33	32.03
2023	HNeRV [21]	33.36	33.62	39.17	32.31	25.60	34.90	28.33	32.47
	DNeRV [22]	33.91	33.17	38.22	32.01	25.56	33.58	27.15	31.94
	FFNeRV [23]	33.62	33.72	38.82	32.80	25.82	34.19	28.37	32.48
	Gomes et al. [24]	33.46	33.76	38.70	28.89	23.88	34.32	27.66	31.52
2024	HiNeRV [27]	34.08	38.68	39.71	36.10	31.53	35.85	30.95	35.27
	NeRV++ [28]	33.57	34.59	38.76	32.46	25.29	34.11	27.65	32.35
	NeRV-BOOST [29]	33.55	34.51	39.04	32.82	26.08	34.54	28.76	32.76
	E-NeRV-BOOST [29]	33.75	35.62	39.61	32.39	27.75	35.48	29.23	33.40
	HNeRV-BOOST [29]	33.80	36.11	39.65	34.28	28.19	35.88	29.33	33.89
	VQ-NeRV [30]	34.15	36.89	39.41	36.77	31.74	36.08	32.85	35.41
	T-NeRV [31]	34.34	35.65	40.26	34.80	28.24	35.25	29.46	34.00
	QS-NeRV [34]	33.92	35.74	39.27	33.83	27.67	35.04	29.97	33.63
	SNeRV [35]	34.14	<u>37.64</u>	39.11	36.17	33.21	34.41	<u>32.71</u>	<u>35.34</u>

Note: The best result for each video is marked in bold. The second-best result for each video is underlined. Results are obtained from the respective papers.

increase of 0.92dB in PSNR w.r.t. NeRV [14] on the UVG dataset [15], as can be seen in Table 2. A second advantage of this approach is that the architecture of the model does not have to change when adapting to different spatial resolutions [19]. HiNeRV [27] additionally uses overlapping patches to avoid spatial inconsistencies between neighboring patches. NIRVANA [19] and C3 [25] use three-dimensional spatiotemporal patches to also exploit temporal redundancies within each patch.

2.2 Model Scope

Model scope refers to the total amount of data captured by each θ_n . The scope of each individual model is therefore inversely proportional to the number of models N used to encode V .

2.2.1 Full Scope

Most works [14, 16–18, 20–24, 27–31, 33, 35, 36] do not explicitly limit the scope, but instead encode the entire input video with a single model. D-NeRV² [20] is specifically aimed at encoding longer and more diverse videos, by conditioning on keyframes.

2.2.2 GoP Scope

NIRVANA [19], on the other hand, trains a separate network per Group of Pictures (GoP). In this way, each subsequent set of parameters θ_n can be fine-tuned from θ_{n-1} , which reduces training time.

2.2.3 Instance Scope

At the extreme end of the spectrum, COOL-CHIC video [26] trains a separate model for each frame. This results in an extremely low decoding complexity of only 900 multiplications per decoded pixel, and a lower decoding delay since each frame is compressed individually. When combined with a single-frame instance size, the downside of this approach is that temporal redundancies are no longer implicitly captured by the model, but must be exploited through some other means. COOL-CHIC video [26] and PNVC [37] are based on a more traditional method by generating I-, P-, and B-frames. Similarly to NIRVANA [19], PNVC additionally fine-tunes the parameters for each frame from those of the previous frame to reduce encoding complexity even further. C3 [25] uses a single-instance scope in combination with three-dimensional spatiotemporal patches.

2.2.4 Layer Scope

QS-NeRV [34] takes a layered approach inspired by scalable video coding [39]. A base layer network based on HNeRV [21] is first used to encode a low-quality version of V . Several enhancement layer networks then use feature maps from the previous layer to decode a residual that can be added to the output of the previous layer.

²Note the distinction between D-NeRV [20] and DNeRV [22], and between T-NeRV [31] and TNeRV [33]

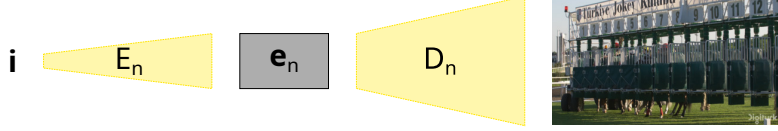


Fig. 1 Conventional NVR architecture, where instances are generated from an input coordinate

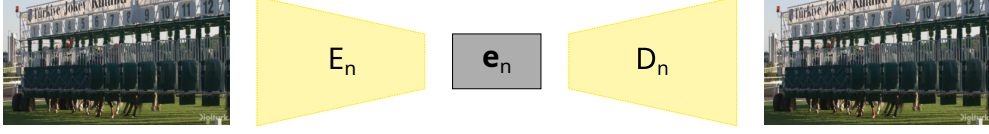


Fig. 2 Hybrid NVR architecture that combines INR- and autoencoder-based methods

2.3 Input Embedding

Instead of decoding each instance from the input coordinate $\mathbf{i}$ directly, the embedding function E_n from Equation (1) first converts $\mathbf{i}$ to some low-dimensional embedding $\mathbf{e}_n$ that is used as input to D_n , as depicted in Figure 1. The resulting embedding is usually in the form of a feature grid with a low spatial dimension but a high number of channels per pixel, which can then be passed to a fully convolutional decoder D_n .

2.3.1 Positional Encoding (PE)

In the original approach proposed by NeRV [14], frames are generated purely based on a frame index interpolated between 0 and 1. This frame index is first encoded using positional encoding [40]

$$\text{PE}(t) = (\sin(b^0 \pi t), \cos(b^0 \pi t), \dots, \sin(b^{l-1} \pi t), \cos(b^{l-1} \pi t)), \quad (3)$$

where l is the length of the embedding and b is a constant factor, commonly set to 1.25. Mapping the input to a higher-dimensional space with high-frequency functions allows the network to better learn high-frequency information than when the frame index is taken directly as input [14, 40–42]. A multilayer perceptron (MLP) is then used to transform the input embedding to a feature grid as input to subsequent convolutional and upsampling layers. This form of input embedding was also adopted by PS-NeRV [18], D-NeRV [20] and NeRV++ [28].

2.3.2 Spatial Grid

A common critique on the combination of PE and MLP described above is that the MLP contains a disproportionate number of parameters compared to the rest of the network [16, 24]. E-NeRV [16] uses a much smaller MLP to generate a one-dimensional temporal feature vector $\mathbf{e}_{n,t} \in \mathbb{R}^d$, combined with a grid of spatial coordinates that is also encoded with PE into a spatial feature grid $\mathbf{e}_{n,s} \in \mathbb{R}^{w \times h \times d}$, with $w < W$ and $h < H$. The spatial and temporal features are then combined through per-feature element-wise multiplication. The resulting feature grid can be passed to D_n directly. Gomes et al. [24] bypass the need for MLP layers in NeRV completely by generating a grid based

on spatial and temporal coordinates and then applying PE to each element of that grid. Although a grid-based embedding reduces or bypasses the reliance on an expensive MLP, this is still a content-agnostic embedding, meaning that content-specific features are encoded solely by the network layers.

2.3.3 Multi-resolution Temporal Grid (MTG)

FFNeRV [23] and HiNeRV [27] use K learned feature grids $\{\mathbf{G}_k \in \mathbb{R}^{T_k \times W_g \times H_g \times C_g}\}_{k=1}^K$ with different temporal resolutions T_k , spatial resolution $W_g \times H_g$ and C_g features. Given a temporal coordinate t , a temporal feature is interpolated from each feature grid, and the results are concatenated along the channel dimension. The concatenated temporal features then serve as input to D_n . Due to interpolation, each of the T_k slices of the MTG affects several adjacent video frames if $T_k < T$. In other words, this type of embedding inherently models temporal redundancies [31]. Furthermore, because the MTG is learned during the training process, this can be regarded as a content-specific embedding.

2.3.4 Instance Embedding (IE)

CNeRV [17] and HNeRV [21] formulate E_n and D_n as encoder and decoder components of an autoencoder, as depicted in Figure 2. When both parts are trained jointly, this essentially allows the network to learn the best input embedding autonomously, instead of using a hand-crafted one. This results in a hybrid between autoencoder-based compression models such as DVC [4], and INR-based methods. Since $\mathbf{e}_n$ is generated from the actual video, it cannot be generated during the decoding process. This means that each IE has to be encoded in the bitstream along with the model weights, resulting in a trade-off between the two. By using a learned embedding, HNeRV [21] achieves an average increase of 1.36dB in PSNR w.r.t. NeRV [14] on the UVG dataset [15], as can be seen in Table 2. DNeRV² [22], QS-NeRV [34], TNeRV² [33], HFS-HNeRV [36], and SNeRV [35] also follow this hybrid approach. VQ-NeRV [30] and PNVC [37] further extend the hybrid architecture similarly to U-shaped networks [43], by producing instance embeddings between each downsampling stage and its corresponding upsampling stage.

IE can be used alongside other embedding types to introduce more spatial or temporal context. HNeRV-BOOST [29] improves HNeRV [21] by conditioning on a positionally encoded temporal index in each upsampling block, as we will explain in more detail in Section 3.3. TNeRV [31] combines frame embeddings with the MTG embedding introduced by FFNeRV [23], based on the earlier observation that the MTG can model temporal redundancies, and also conditions each upsampling block on a positionally encoded temporal index.

2.4 Temporal Context

Traditional video codecs employ motion estimation and motion compensation techniques to explicitly model temporal redundancies in an input video. Although INRs are able to encode redundancies into the network’s parameters implicitly, some works

explicitly exploit temporal information to achieve higher temporal consistency in the decoded output.

2.4.1 Motion Estimation and Compensation (ME/MC)

Some NVRs still perform ME/MC explicitly. D-NeRV [20] performs ME/MC with features extracted from key frames at multiple scales. As mentioned in Section 2.2, this technique allows the model to perform significantly better on longer and more diverse videos. FFNeRV [23] and T-NeRV² [31] generate independent reference frames, along with flow maps and interpolation weights, which are used to warp and interpolate features from adjacent independent frames into a final reconstructed frame. This has been shown to perform better on videos with highly dynamic content. Similarly, COOL-CHIC video [26] generates flow maps, interpolation weights, and residuals that are fed to an inter coding module.

2.4.2 Temporal Embedding (TE)

Some hybrid NVRs use adjacent frames to generate temporal embeddings alongside the instance embedding. DNeRV [22] encodes the differences between the current frame and the previous and next frames. Similarly, TNeRV [33] introduces a temporal diversity exploration module with 3D convolutions in the encoder to extract temporal information. SNeRV [35] uses embeddings from adjacent frames during the upsampling stages.

3 Low-level Network Design

The NeRV [14] decoder consists of five NeRV blocks that gradually upsample the input to obtain a final reconstructed frame. Each NeRV block uses a 3×3 subpixel convolution, consisting of a convolutional and a pixelshuffle layer [44], to scale the spatial resolution of the input feature map by a factor S . The first block increases the number of channels by some expansion factor, while subsequent blocks reduce the number of channels by a factor of 2. Each block is followed by a GELU activation function. Common upsampling factors are (5, 2, 2, 2, 2) for 720p resolution and (5, 3, 2, 2, 2) for 1080p resolution. Specifically, for an initial channel dimension C_{in} and a target channel dimension C_{out} , the NeRV block is shown in Figure 3(a). The complete decoder architecture for 1080p resolution is shown in Figure 4.

There are two common concerns with this architecture. First, the upsampling blocks have an uneven and inefficient distribution of parameters [16, 21, 23, 27]. Section 3.1 discusses how the decoder blocks can be designed to address this issue. As a general rule, when a large number of parameters can be eliminated at the cost of a slight decrease in performance, the saved parameter budget can be used to increase the channel dimension of each intermediate feature map, thereby increasing the representation capacity of the network as a whole. This is supported by Figure 7, which shows that simply increasing the number of parameters in the same model significantly improves the reconstruction quality.

The second common concern with this architecture is that it often fails to preserve high-frequency details as a result of convolution operations [35, 36], causing a spectral

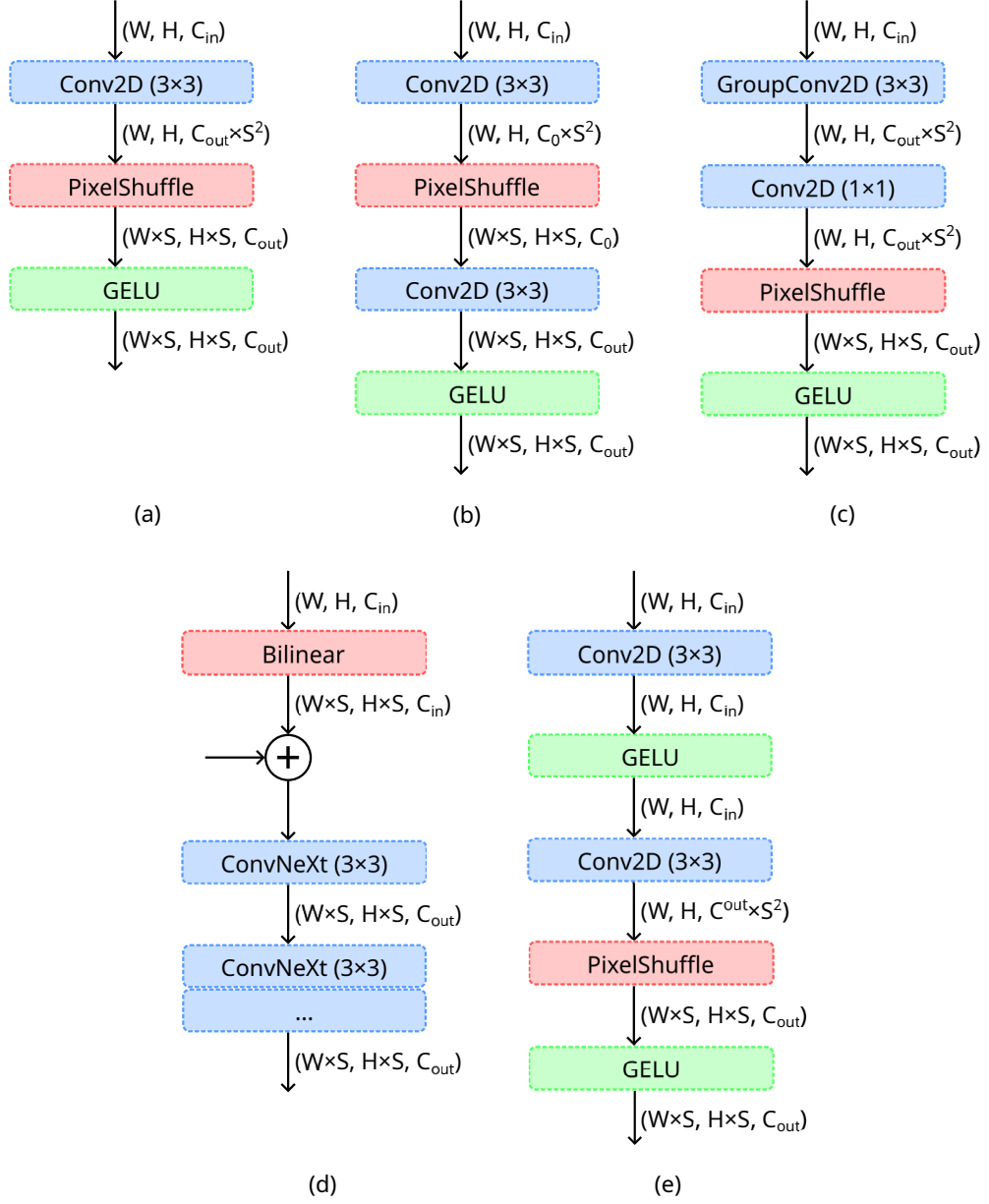


Fig. 3 Upsampling block architectures (a) NeRV block (b) First upsampling block in E-NeRV, with $C_0 = \min(C_{in}, C_{out})/4$ (c) FFNeRV block in stages 2-5 (d) HiNeRV block (e) T-NeRV block in stages 3-5

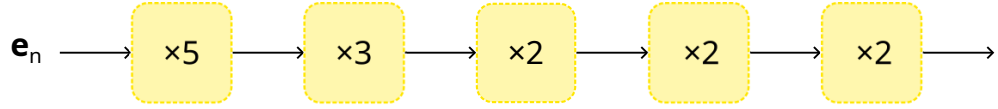


Fig. 4 NeRV decoder architecture for 1080p resolution

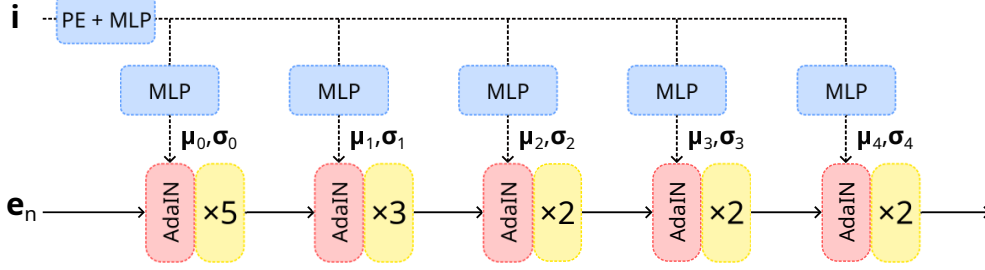


Fig. 5 Content-agnostic temporal information fusion with AdaIN

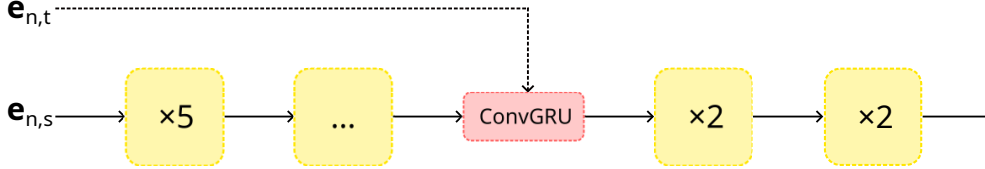


Fig. 6 Temporal information fusion with ConvGRU

bias towards lower frequency components. In fact, the spectral bias issue is not exclusive to NVR, but also occurs in other applications of neural networks [41]. Section 3.2 explains how the spectral bias issue can be alleviated by adding inductive biases towards high-frequency information to the decoder. Section 3.3 then explores several ways to incorporate temporal information in the upsampled feature maps. For hybrid approaches, the encoder architecture also plays a role in compression performance. This is discussed in section 3.4. Finally, Section 3.5 discusses several loss functions that are used for training NVRs, and how these can also be used towards solving the spectral bias issue.

3.1 Upsampling Architecture

The convolutional layer in each NeRV block contains $C_{in} \times S^2 \times C_{out}$ kernels of size $k \times k$, resulting in a total of $k^2 \times C_{in} \times S^2 \times C_{out}$ parameters. This is mostly problematic in the first upsampling block, where $S = 5$. E-NeRV [16] improves this by introducing an intermediate channel dimension $C_0 = \min(C_{in}, C_{out})/4$ in the first upsampling block. The improved block structure is shown in Figure 3(b). This results in two convolutional layers with $k^2 \times C_{in} \times S^2 \times C_0$ parameters and $k^2 \times C_0 \times C_{out}$ parameters, respectively, or a total parameter count of $k^2 \times C_0 \times (C_{in} \times S^2 + C_{out})$ for the first upsampling block. This amounts to at least four times fewer parameters than the NeRV block if $C_{in} \leq C_{out}$. A detailed calculation of this ratio can be found in [16].

FFNeRV [23] follows yet another approach by substituting the convolutional layer for a depthwise separable convolution [45], composed of a grouped convolution followed by a pointwise convolution, as shown in Figure 3(c). With g channel-wise groups that are processed separately, the first convolutional layer has a parameter count of $k^2 \times \frac{C_{in} \times S^2 \times C_{out}}{g}$. The point-wise convolution, equivalent to a pixel-wise linear layer,

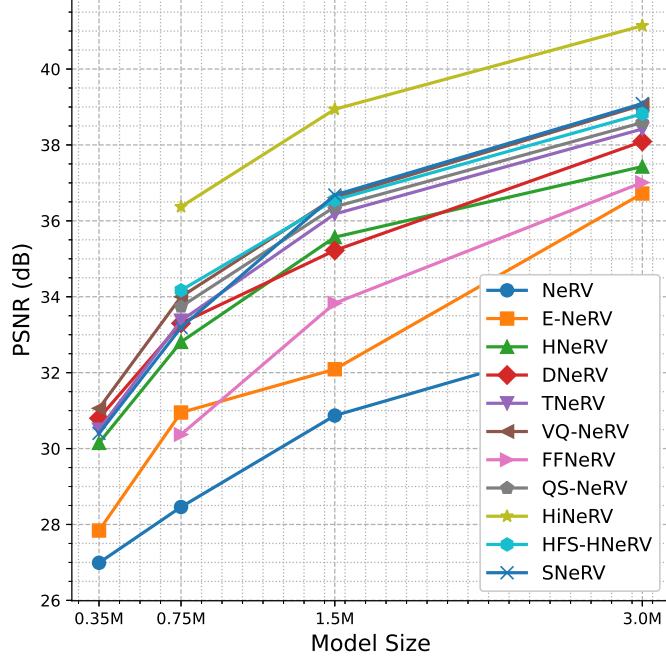


Fig. 7 PSNR results on Big Buck Bunny (132 frames) with different model sizes (in millions of parameters), as obtained from the respective papers

then contains $S^4 \times C_{out}^2$ parameters. Together, this results in a ratio of $\frac{1}{g} + \frac{S^2 \times C_{out}}{k^2 \times C_{in}}$ with respect to the NeRV block. From this ratio, we can see that when $S > k$ and $C_{out} \geq C_{in}$, the depthwise separable convolution actually contains more parameters than a regular convolution. Therefore, this structure is only used in the second upsampling block and onward, where $S \leq k$ and $C_{out} \leq C_{in}$.

Instead of subpixel convolution, HiNeRV [27] uses a much simpler bilinear interpolation for upsampling. A local feature grid similar to the multiresolution temporal embeddings is then added to the upsampled feature map, in order to preserve high-frequency details that would otherwise be lost due to interpolation. The result is finally passed through a series of ConvNeXt [46] blocks. The activation layer at the end of the block is omitted since each ConvNeXt block already contains an activation layer. The architecture of the HiNeRV block is illustrated in Figure 3(d).

HNeRV [21] does not change the architecture of the upsampling blocks, but instead varies the kernel size from 1 in the first upsampling stage, to 3 in stage 2, to 5 in stages 3 and onward. Furthermore, the channel dimension is reduced by a factor of 1.2 in each layer, instead of 2 as in the original NeRV [14] block. This combination of larger convolution kernels and a smaller channel dimension scale factor places more importance in the later stages of the network, allowing for a more even distribution of parameters across the network. TNeRV [33] uses the same upsampling block architecture as HNeRV. T-NeRV [31] also uses a similar block architecture, but the 5×5 convolutions are replaced by two 3×3 convolutions and an additional activation layer, as in Figure

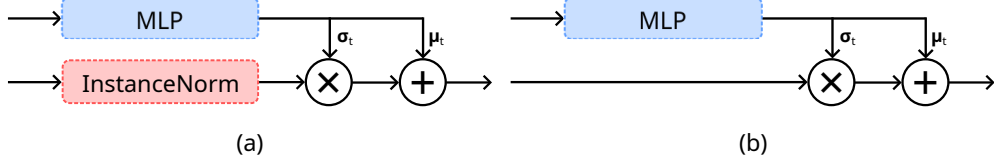


Fig. 8 Temporal information fusion blocks (a) AdaIN block (b) Affine transformation used by HNeRV-BOOST

3(e). This significantly reduces the number of parameters, while still maintaining the same spatial range as the original 5×5 convolution.

More recently, PNVC [37] has introduced weight reparameterization and knowledge distillation schemes in its upsampling blocks to reduce decoding complexity. Weight reparameterization was originally introduced in [47], where complex layers are substituted with simpler operations during inference. Weight reparameterization is implemented by decomposing a single fully connected layer into several parallel stacks of fully connected layers. Similar to weight reparameterization, knowledge distillation schemes train a small student model to mimic a larger teacher model [48]. This is implemented in PNVC with a mask decaying strategy [49, 50], where a mask M is initialized as **1** and gradually decayed towards **0** during training. This can be interpreted as an interpolation weight that gradually pushes more importance towards the student model. A detailed description of the methods used by PNVC can be found in [37].

3.2 Preservation of High-frequency Components

In order to address the issue of spectral bias mentioned in the introduction of this section, HFS-HNeRV [36] extends the HNeRV [21] block with a high-frequency spectrum convolution module that extracts more high-frequency features. Feature maps are first transformed with the discrete wavelet transform (DWT). The DWT decomposes a feature map into four sets of wavelet coefficients C_{LL} , C_{LH} , C_{HL} , and C_{HH} , with low-frequency components, and horizontal, vertical, and diagonal high-frequency features, respectively. The input feature maps are then augmented with frequency-domain features extracted from the wavelet coefficients.

In contrast, SNeRV [35] uses a small network to extract the C_{LH} , C_{HL} and C_{HH} components after the final upsampling stage, and then performs an inverse DWT to restore the final frame. The final upsampled feature map serves as the C_{LL} component in the IDWT, and also as input to extract the aforementioned high-frequency components. Table 2 shows that on average, SNeRV achieves an increase of almost 3dB in PSNR w.r.t. HNeRV, on which it is based.

3.3 Temporal Information Fusion

Note from Section 3.1 that the upsampling operations used by NeRV [14] exploit spatial information exclusively, whereas temporal information is only introduced via input embedding. Further research has shown that the network’s performance can be improved by fusing temporal information in the decoder. Inspired by style transfer networks, E-NeRV [16] and T-NeRV² [31] both use an AdaIN [51] layer at the start

of each upsampling block to fuse a temporal feature vector with each intermediate feature map. AdaIN combines instance normalization with an affine transformation, and is defined as

$$\text{AdaIN}(\mathbf{x}_t, \boldsymbol{\mu}_t, \boldsymbol{\sigma}_t) = \boldsymbol{\sigma}_t \left(\frac{\mathbf{x}_t - \boldsymbol{\mu}(\mathbf{x}_t)}{\boldsymbol{\sigma}(\mathbf{x}_t)} \right) + \boldsymbol{\mu}_t, \quad (4)$$

where $\boldsymbol{\mu}(\mathbf{x}_t)$ and $\boldsymbol{\sigma}(\mathbf{x}_t)$ are the channel-wise mean and variance extracted from $\mathbf{x}_t$, and $\boldsymbol{\mu}_t$ and $\boldsymbol{\sigma}_t$ are channel-wise shift and scale parameters extracted from a positionally encoded temporal feature vector through a small MLP layer, as shown in Figure 8(a). The complete decoder architecture for E-NeRV is shown in Figure 5. PS-NeRV [18] follows a similar approach, but the AdaIN layer is applied after the upsampling step instead. Because AdaIN performs normalization, which is often used to prevent overfitting, Zhang et al. [29] argue that this might not be ideal for INR. HNeRV-BOOST [29] instead performs a similar operation to AdaIN, but without the instance normalization step, resulting in a simple affine transformation as shown in Figure 8(b).

Instead of using purely content-agnostic temporal features, some hybrid approaches use a separate encoder branch to generate content-specific temporal embeddings, as summarized in Section 2.4. DNeRV [22] and TNeRV [33] treat this temporal embedding as the hidden state in a recurrent neural network (RNN) by fusing the spatial and temporal embeddings in a convolutional gated recurrent unit (ConvGRU) [52]. Additionally, TNeRV first modulates both spatial and temporal embeddings with a content-agnostic temporal feature, similar to the methods described earlier in this section.

3.4 Encoder Architecture

For hybrid approaches, encoder architecture also plays a role in compression performance. CNeRV [17] uses a tiny single-layer encoder, while HNeRV [21], as well as the base layer of QS-NeRV [34], use a slightly larger encoder architecture composed of five ConvNeXt [46] blocks. Because the encoder can be discarded after training, a larger encoder does not increase the size of the bitstream, but can potentially generate more powerful embeddings. Based on this observation, T-NeRV [31] uses a much larger ConvNeXt encoder to generate frame embeddings. A downside of this approach is that a larger encoder also increases the computational cost and energy usage during training.

The temporal difference encoder employed by DNeRV [22] is also built of a sequence of ConvNeXT blocks that encode the differences between the previous and current frame, and the current and next frame. TNeRV [33] uses a slightly more complex temporal diversity encoder that first extracts features from each frame separately with a series of ConvNeXT blocks. The extracted features are then concatenated and processed by a 3D convolutional layer. The result is finally encoded by a series of ConvNeXT blocks into a temporal diversity feature map.

3.5 Loss Function

NeRV [14] is trained with a linear combination of spatial-domain L1 and SSIM losses, with a trade-off parameter α commonly set to 0.7:

$$L(\mathbf{V}, \mathbf{i}; \Theta) = \alpha \|\mathbf{v}_i - \mathcal{F}(\mathbf{i}; \Theta)\| + (1 - \alpha)(1 - \text{SSIM}(\mathbf{v}_i, \mathcal{F}(\mathbf{i}; \Theta))), \quad (5)$$

where $\mathbf{v}_i$ represents the region of the ground-truth video $\mathbf{V}$ indexed by $\mathbf{i}$ and $\mathcal{F}(\mathbf{i}; \Theta)$ is the prediction of the same region of $\mathbf{V}$.

Several works have addressed the spectral bias issue discussed in Section 3.2 by incorporating frequency domain terms in the loss function. In addition to the spatial L1 loss, HNeRV-BOOST [29] also adds a frequency domain L1 loss term:

$$\begin{aligned} L(\mathbf{V}, \mathbf{i}; \Theta) = & \|\text{FFT}(\mathbf{v}_i) - \text{FFT}(\mathcal{F}(\mathbf{i}; \Theta))\| \\ & + \lambda \alpha \|\mathbf{v}_i - \mathcal{F}(\mathbf{i}; \Theta)\| \\ & + \lambda (1 - \alpha)(1 - \text{SSIM}(\mathbf{v}_i, \mathcal{F}(\mathbf{i}; \Theta))), \end{aligned} \quad (6)$$

where λ is another hyperparameter to balance the trade-off between the different parts of the loss function. HFS-HNeRV [36] addresses the same issue by formulating the loss function as a combination of mean squared error (MSE) loss in the spatial domain, and the MSE of a high-pass-filtered version of the reconstructed and ground truth frames. SNeRV [35] follows yet another approach by calculating the loss function from (5) over the spatial images, as well as the four wavelet coefficients.

4 Model Compression

By encoding a video into the parameters of a neural network, NVR-based codecs convert the video compression problem to a model compression problem. NeRV [14] performs video compression in four sequential steps: training, model pruning, weight quantization, and entropy coding. This is still very much a hand-crafted compression pipeline, where the training step can be seen as a transformation from one domain to another, not unlike the discrete cosine transform in the traditional image and video compression pipeline. This raises the need for more advanced model compression techniques. We start by discussing weight pruning and quantization techniques in Sections 4.1 and 4.2, respectively. Section 4.3 finally discusses how rate-distortion optimization can be used to build a fully end-to-end optimized compression scheme. Table 3 provides a brief overview of works with an explicit focus on model compression.

4.1 Pruning

NeRV [14], as well as several consecutive works [16–18, 20–23, 30, 33–35], uses a simple global unstructured pruning method, where all weights smaller than some threshold θ_q are pruned, with θ_q chosen as the q -th percentile value over all weights in θ_n . HiNeRV [27] uses a slightly more advanced pruning method, where parameters in larger layers have a higher chance to be pruned than parameters in smaller layers.

Table 3 Overview of compression methods used in NVRs

Year	Name	Pruning	Quantization
2021	NeRV [14]	✓	SQ (Fixed)
2023	NIRVANA [19]	✓	SQ (Learned)
	FFNeRV [23]		QAT
	Gomes et al. [24]		SQ (Learned)
	C3 [25]		SQ (Learned)
2024	COOL-CHIC video [26]	✓	SQ (Learned)
	HiNeRV [27]		QAT
	HNeRV-BOOST [29]		SQ (Learned)
	VQ-Nerv [30]	✓	VQ
	T-NeRV [31]		SQ (Learned)
	NVRC [32]		SQ (Learned)

Note: This table only includes works with an explicit focus on model compression.

This is based on the intuition that larger layers contain more redundancy, and thus pruning has less negative impact than in smaller layers.

More advanced pruning methods such as filter pruning [53] and structured sparsity [54–56] exist, both of which can also reduce computational cost. To the best of our knowledge, these methods have not yet been explored in the context of NVR.

4.2 Quantization

Since entropy coding relies on a discrete set of values to be encoded, the continuous network weights and embeddings have to be quantized before encoding.

4.2.1 Scalar Quantization (SQ)

Early works on NVR [14, 17, 20–22, 33–35] use a fixed SQ step after training to quantize each weight or embedding to a fixed number of bits. SQ is implemented as an affine transformation with scale and shift parameters α and β so that

$$Q(\mathbf{x}) = \left\lfloor \frac{\mathbf{x} + \beta}{\alpha} \right\rfloor, \quad (7)$$

$$Q^{-1}(\mathbf{x}) \approx \mathbf{x} \times \alpha - \beta, \quad (8)$$

where $\lfloor \cdot \rfloor$ represents the rounding operator. As a result of rounding, the inverse quantization function Q^{-1} is always an approximation of the original value.

4.2.2 Vector Quantization (VQ)

VQ [57] presents a more general approach to quantization by grouping continuous values into several clusters, for example with k-means clustering. Each value is then represented by the corresponding cluster centroid. VQ-NeRV [30] implements VQ by discretizing residual embeddings in a codebook. However, because of the higher complexity of VQ, SQ is often the preferred method.

4.2.3 Quantization Aware Training (QAT)

Instead of performing quantization as a separate step after training, several works [58, 59] have shown that the quantization error can be reduced by applying some quantization method during the training process. With QAT, the model weights are quantized and rounded before each forward pass. This approach was integrated into NVR in combination with scalar quantization by FFNeRV [23] and HiNeRV [27].

A common difficulty with quantization during training is that the gradient of the rounding operation is zero almost everywhere. This is usually avoided in one of two ways. The *straight-through estimator* (STE) [60] applies rounding during the forward pass, while acting like an identity operation during gradient computation. Alternatively, adding a *uniform noise term* with the same width as the quantization bins to the weights during training can be used as an approximation for the quantization error [61]. FFNeRV [23] uses the STE, while HiNeRV [27] uses a noise term.

4.3 Rate-Distortion Optimization

With fixed quantization methods such as those described in Section 4.2, finding the right quantization parameters for some rate-distortion objective always requires manual fine-tuning. Furthermore, it can be seen from Table 2 that there is no single model architecture that performs best for every possible video. Similar to picking the right quantization parameters, it is infeasible to manually tune the model architecture for a specific video. Both of these problems can be solved by treating NVR as a rate-distortion optimization (RDO) problem where a decrease in distortion always results in an increase in bitrate, and vice versa. This translates to a loss function of the form

$$L_{RDO} = L_{\text{distortion}} + \lambda L_{\text{rate}}, \quad (9)$$

where $L_{\text{distortion}}$ represents any measure of distortion as outlined in Section 3.5, L_{rate} represents an approximation of the length of the bitstream encoding the network parameters θ_n and embeddings e_n , and λ is a hyper-parameter that controls the trade-off between the two.

4.3.1 Rate Minimization for Latent Representations

RDO has been widely used in autoencoder-based compression models for images [62, 63] and videos [4–6], where the input is first converted to a latent representation $\mathbf{y}$, which is then quantized to $\hat{\mathbf{y}}$ and encoded with entropy coding. According to Shannon’s source coding theorem [64], the minimal length of a bit stream encoding $\hat{\mathbf{y}}$ is equal to the entropy of $\hat{\mathbf{y}}$, based on the underlying probability distribution p . In practice, p is unknown but can be approximated by some estimate q_ϕ of the true probability distribution, with a separate set of parameters ϕ . Therefore, the rate and distortion can be optimized jointly by defining L_{rate} as the cross-entropy between p and q_ϕ ,

$$L_{\text{rate}} = \mathbb{E}_{\hat{\mathbf{y}} \sim p}[-\log_2 q_\phi(\hat{\mathbf{y}})]. \quad (10)$$

C3 [25] and COOL-CHIC video [12] incorporate this method into an NVR scheme to minimize the rate consumed by the instance embeddings. However, both models fail to include the rate penalty caused by the model parameters in the loss function, causing them to not be fully end-to-end optimized.

4.3.2 Rate Minimization for Model Parameters

Oktay et al. [65] were the first to adapt RDO to a general model compression scheme. During each forward pass, a learned transformation f_ϕ is applied to decode the model parameters θ from a quantized latent space $\hat{\theta}$. Since f_ϕ is used as an inverse quantization function, Gomes et al. [24], NIRVANA [19], HNeRV-BOOST [29] and T-NeRV [31] all formulate f_ϕ as in Equation (8), but more complex transformations are also possible [32]. Similar to the quantized latent representation $\hat{y}$ in [62] and Equation (10), a learned entropy model q_ϕ is used to tune f_ϕ to the rate-distortion objective. This results in two additional sets of parameters, $\phi = \{\phi_{\text{quant}}, \phi_{\text{em}}\}$. Finally, $\hat{\theta}$ is entropy coded and transmitted along with ϕ .

Recall from Section 4.2 that quantization uses the non-differentiable rounding operator. When calculating the rate term, the rounding error is approximated by a noise term. When calculating the distortion term, the STE is used in order to avoid using noisy parameters for inference.

In practice, separate quantization and entropy models are learned for each layer of the network to allow the entropy model to better capture the distribution of each parameter group independently. This also allows the model to autonomously assign more or fewer bits to different parts of the network, based on the encoded content. For example, experiments by the authors of both T-NeRV [31] and NVRC [32] show that multiresolution temporal grid embeddings take up more bits per parameter for dynamic content than for static content to better exploit their temporal information. This suggests that RDO makes it possible to build a single NVR model that can automatically tune itself to many diverse input videos.

NVRC [32] extends the reparameterization approach with more complex quantization and entropy models. Because this results in more overhead from encoding ϕ , ϕ itself is quantized as $\hat{\phi} = \{\hat{\phi}_{\text{quant}}, \hat{\phi}_{\text{em}}\}$ and encoded with another set of learned quantization and entropy models parameterized by $\psi = \{\psi_{\text{quant}}, \psi_{\text{em}}\}$. Finally, ψ is simply quantized into full/half precision as $\hat{\psi} = \{\hat{\psi}_{\text{quant}}, \hat{\psi}_{\text{em}}\}$. This more complex coding structure makes NVRC the first NVR-based codec to outperform VVC VTM [1] on the UVG dataset [32].

5 Practical Considerations for Video Streaming

Now that we have reviewed the existing techniques for NVR, we wish to identify common weaknesses and opportunities for future work. We will consider potential applications in an over-the-top (OTT) video streaming context, in which video compression plays a crucial role. Several factors must be considered before NVR-based video codecs can be deployed in a practical video streaming scenario. Adaptive bitrate (ABR) streaming paradigms such as MPEG-DASH [66] require videos to be split into short segments and encoded at several fixed bitrates, allowing streaming clients to

select the appropriate bitrate for the current network conditions. Other obvious factors that determine the final quality of experience (QoE) are video reconstruction quality, decoding complexity, and, in the case of live streaming, encoding delay. The first requirement is easy to satisfy with an NVR-based codec, since NVR models are best trained on short video segments. However, all model parameters must be downloaded before the first frame can be decoded, causing high latency when the scope is large. In other words, it is beneficial to use a smaller scope, or even a single-frame scope as proposed by COOL-CHIC video [26] and PNVC [37] for streaming. Sections 5.1, 5.2, 5.3, and 5.4 discuss common challenges related to the requirements of reconstruction quality, rate control, decoding complexity, and encoding complexity, respectively.

5.1 Spectral Bias

The spectral bias towards lower frequencies is the most common issue related to the reconstruction quality of NVRs and INRs in general. We already discussed several different methods to alleviate the spectral bias issue throughout this paper. PS-NeRV [18] has shown that a patch-wise approach generally preserves more high-frequency features than a frame-wise approach (Section 2.1.3). HiNeRV [27] additionally adds high-frequency information after each upsampling step (Section 3.1). HFS-HNeRV [36] and SNeRV [35] use wavelet transforms in the decoder network to add inductive biases towards high-frequency components (Section 3.2). Lastly, several works also add frequency-domain components to the loss function [29, 35, 36] (Section 3.5). For general INRs, Shi et al. [67] have introduced a weight reparameterization scheme that can also alleviate the spectral bias issue. Since spectral bias is inherently related to neural networks [41], NVR models should always compensate for this explicitly.

5.2 Rate Control

For ABR streaming to work correctly, each segment must be encoded at fixed bitrates. We have already discussed several levers that can control the final bitrate of an NVR model. On the one hand, the number of parameters in the model can be adjusted manually, for example, by changing the number of channels in each intermediate feature map (Section 3 and Figure 7). On the other hand, the rate control parameter λ can adjust the weight of the distortion term in the rate-distortion objective (Section 4.3). Increasing the scope of each model also reduces the bitrate but causes higher latency for streaming (Section 2.2). Future research should clarify how the best configuration can be found for any combination of input video and target bitrate.

5.3 Decoding Complexity

Since videos are decoded on the (potentially mobile) client device, decoding complexity plays a vital role in the final QoE. Existing works have focused mostly on designing more parameter-efficient models to increase representation capacity, and on compression methods with a focus on reducing the footprint of stored model weights. Future work should also focus on incorporating existing methods for complexity reduction in the NVR pipeline. As mentioned in Section 4.1, advanced pruning methods such as filter pruning and structured sparsity are still underexplored for NVR models.

Girish et al. [56] have already shown how structured sparsity can be used in a scheme that minimizes rate and computational cost jointly. Other techniques for reducing model complexity include weight reparameterization and knowledge distillation, both of which we briefly discussed in section 3.1. PNVC [37] incorporates both of these techniques in an NVR framework. Unfortunately, both reparameterization and knowledge distillation schemes increase the computational complexity during training.

5.4 Encoding complexity

For live streaming, encoding complexity is of even greater importance than for non-live streaming since training NVR models consumes considerable time, computational resources, and energy. When training multiple subsequent NVR models, encoding time can be reduced by fine-tuning each set of parameters θ_n from the previous parameters θ_{n-1} [19, 37]. Alternatively, hyper-learning has been applied to general INRs [68, 69], as well as NeRV [70] to speed up encoding. Instead of optimizing the model weights with gradient descent, a hyper-network is trained to predict the model weights directly for some input video. Hyper-learning significantly reduces encoding times, but also reintroduces the reliance on large datasets for training. Further research is needed to investigate how hyper-learning can be applied to more complex, end-to-end optimized NVR pipelines. Finally, research in areas outside INR has shown that the computational complexity of neural networks can be reduced by using weights in 16-bit [71] or even 8-bit [58] precision during training, with little impact on performance. With many optimization strategies still underexplored, future research has yet to point out whether real-time encoding with NVR-based codecs is achievable.

6 Conclusion

NVR-based video codecs present an alternative to both traditional video codecs and autoencoder-based learned video compression schemes. Learned video compression schemes generally have more potential for optimization than traditional video codecs because they do not rely on hand-crafted heuristics. Because of their overfitting nature, NVR-based codecs can be fine-tuned for specific videos without reliance on large datasets and have a lower decoding complexity compared to autoencoder-based compression schemes. We have provided a systematic overview of the representative works, and discussed several techniques related to model design and compression. To identify common weaknesses, we considered a potential application in the context of OTT video streaming. Specifically, we discussed issues surrounding spectral bias, rate control, and computational complexity. Those common weaknesses provide interesting avenues to focus on in future work, to increase the practical applicability of NVR-based codecs.

Declarations

Ethical approval. The authors declare that this work has not been published elsewhere, and has not been submitted to more than one journal for simultaneous consideration.

Consent to participate. All authors consent to submit this work in its current form.

Consent to publish. All authors consent to publish this work.

Author contributions. Hannes Keunen: Literature review, writing original draft. Maarten Wijnants and Jori Liesenborgs: Review and editing.

Funding. This work is supported by the Special Research Fund of Hasselt University under grant number BOF24OWB27.

Data availability. Not applicable.

Competing interest. The authors declare that they have no conflict of interest.

References

- [1] B. Bross, Y.-K. Wang, Y. Ye, S. Liu, J. Chen, G. J. Sullivan, and J.-R. Ohm, “Overview of the versatile video coding (VVC) standard and its applications,” *IEEE Trans. Circuits Syst. Video Techn.*, vol. 31, no. 10, pp. 3736–3764, 2021. [Online]. Available: <https://doi.org/10.1109/TCSVT.2021.3101953>
- [2] G. J. Sullivan, J.-R. Ohm, W. Han, and T. Wiegand, “Overview of the high efficiency video coding (HEVC) standard,” *IEEE Trans. Circuits Syst. Video Techn.*, vol. 22, no. 12, pp. 1649–1668, 2012. [Online]. Available: <http://doi.ieeecomputersociety.org/10.1109/TCSVT.2012.2221191>
- [3] F. Pakdaman, M. A. Adelimanesh, M. Gabbouj, and M. R. Hashemi, “Complexity analysis of next-generation VVC encoding and decoding,” in *IEEE Int. Conf. Image Process. (ICIP 2020)*, 2020, pp. 3134–3138. [Online]. Available: <https://doi.org/10.1109/ICIP40778.2020.9190983>
- [4] G. Lu, W. Ouyang, D. Xu, X. Zhang, C. Cai, and Z. Gao, “DVC: An end-to-end deep video compression framework,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR 2019)*, 2019, pp. 11 006–11 015.
- [5] J. Li, B. Li, and Y. Lu, “Neural video compression with feature modulation,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR 2024)*, 2024, pp. 26 099–26 108. [Online]. Available: <https://doi.org/10.1109/CVPR52733.2024.02466>
- [6] F. Mentzer, G. Toderici, D. Minnen, S. Caelles, S. J. Hwang, M. Lucic, and E. Agustsson, “VCT: A video compression transformer,” in *Adv. Neural Inf. Process. Syst. 35 (NeurIPS 2022)*, 2022.
- [7] S. Huo, D. Liu, H. Zhang, L. Li, S. Ma, F. Wu, and W. Gao, “Towards hybrid-optimization video coding,” *ACM Comput. Surv.*, vol. 56, no. 9, 2024. [Online]. Available: <https://doi.org/10.1145/3652148>
- [8] Z. Chen and H. Zhang, “Learning implicit fields for generative shape modeling,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR 2019)*, 2019, pp. 5939–5948.
- [9] L. M. Mescheder, M. Oechsle, M. Niemeyer, S. Nowozin, and A. Geiger, “Occupancy networks: Learning 3D reconstruction in function space,” in *Proc.*

- IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR 2019)*, 2019, pp. 4460–4470.
- [10] J.-J. Park, P. Florence, J. Straub, R. A. Newcombe, and S. Lovegrove, “DeepSDF: Learning continuous signed distance functions for shape representation,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR 2019)*, 2019, pp. 165–174.
 - [11] E. Dupont, A. Goliński, M. Alizadeh, Y. W. Teh, and A. Doucet, “COIN: Compression with implicit neural representations,” *arXiv preprint arXiv:2103.03123*, 2021. [Online]. Available: <https://doi.org/10.48550/arXiv.2103.03123>
 - [12] T. Ladune, P. Philippe, F. Henry, G. Clare, and T. Leguay, “COOL-CHIC: Coordinate-based low complexity hierarchical image codec,” in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV 2023)*, 2023, pp. 13 469–13 476. [Online]. Available: <https://doi.org/10.1109/ICCV51070.2023.01243>
 - [13] V. Sitzmann, J. N. P. Martel, A. W. Bergman, D. B. Lindell, , and G. Wetzstein, “Implicit neural representations with periodic activation functions,” in *Adv. Neural Inf. Process. Syst. 33 (NeurIPS 2020)*, 2020.
 - [14] H. Chen, B. He, H. Wang, Y. Ren, S.-N. Lim, and A. Shrivastava, “NeRV: Neural representations for videos,” in *Adv. Neural Inf. Process. Syst. 34 (NeurIPS 2021)*, 2021, pp. 21 557–21 568.
 - [15] A. Mercat, M. Viitanen, and J. Vanne, “UVG dataset: 50/120fps 4K sequences for video codec analysis and development,” in *Proc. 11th ACM Multimedia Syst. Conf.*, 2020, p. 297–302. [Online]. Available: <https://doi.org/10.1145/3339825.3394937>
 - [16] Z. Li, M. Wang, H. Pi, K. Xu, J. Mei, and Y. Liu, “E-NeRV: Expedite neural video representation with disentangled spatial-temporal context,” in *Proc. 17th Eur. Conf. Comput. Vis. (ECCV 2022)*, 2022, pp. 267–284. [Online]. Available: https://doi.org/10.1007/978-3-031-19833-5_16
 - [17] H. Chen, M. Gwilliam, B. He, S.-N. Lim, and A. Shrivastava, “CNeRV: Content-adaptive neural representation for visual data,” in *Br. Machine Vis. Conf. (BMVC 2022)*, 2022, p. 510.
 - [18] Y. Bai, C. Dong, C. Wang, and C. Yuan, “PS-NeRV: Patch-wise stylized neural representations for videos,” in *IEEE Int. Conf. Image Process. (ICIP 2023)*, 2023, pp. 41–45. [Online]. Available: <https://doi.org/10.1109/ICIP49359.2023.10222144>
 - [19] S. R. Maiya, S. Girish, M. Ehrlich, H. Wang, K. S. Lee, P. Poirson, P. Wu, C. Wang, and A. Shrivastava, “NIRVANA: Neural implicit representations of videos with adaptive networks and autoregressive patch-wise modeling,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR 2023)*, 2023, pp. 14 378–14 387.
 - [20] B. He, X. Yang, H. Wang, Z. Wu, H. Chen, S. Huang, Y. Ren, S.-N. Lim, and A. Shrivastava, “Towards scalable neural representation for diverse videos,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR 2023)*, 2023, pp. 6132–6142. [Online]. Available: <https://doi.org/10.1109/CVPR52729.2023.00594>
 - [21] H. Chen, M. Gwilliam, S.-N. Lim, and A. Shrivastava, “HNeRV: A hybrid neural representation for videos,” in *Proc. IEEE/CVF Conf. Comput. Vis.*

- Pattern Recognit. (CVPR 2023)*, 2023, pp. 10 270–10 279. [Online]. Available: <https://doi.org/10.1109/CVPR52729.2023.00990>
- [22] Q. Zhao, M. S. Asif, and Z. Ma, “DNeRV: Modeling inherent dynamics via difference neural representation for videos,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR 2023)*, 2023, pp. 2031–2040. [Online]. Available: <https://doi.org/10.1109/CVPR52729.2023.00202>
 - [23] J. C. Lee, D. Rho, J. H. Ko, and E. Park, “FFNeRV: Flow-guided frame-wise neural representations for videos,” in *Proc. 31st ACM Int. Conf. Multimedia (MM 2023)*, 2023, pp. 7859–7870. [Online]. Available: <https://doi.org/10.1145/3581783.3612444>
 - [24] C. Gomes, R. Azevedo, and C. Schroers, “Video compression with entropy-constrained neural representations,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR 2023)*, 2023, pp. 18 497–18 506. [Online]. Available: <https://doi.org/10.1109/CVPR52729.2023.01774>
 - [25] H. Kim, M. Bauer, L. Theis, J. R. Schwarz, and E. Dupont, “C3: High-performance and low-complexity neural compression from a single image or video,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR 2024)*, 2024, pp. 9347–9358. [Online]. Available: <https://doi.org/10.1109/CVPR52733.2024.00893>
 - [26] T. Leguay, T. Ladune, P. Philippe, and O. Déforges, “Cool-chic video: Learned video coding with 800 parameters,” in *Proc. Data Compression Conf. (DCC 2024)*, 2024, pp. 23–32. [Online]. Available: <https://doi.org/10.1109/DCC58796.2024.00010>
 - [27] H. M. Kwan, G. Gao, F. Zhang, A. Gower, and D. Bull, “HiNeRV: Video compression with hierarchical encoding-based neural representation,” in *Adv. Neural Inf. Process. Syst. 36 (NeurIPS 2023)*, 2023.
 - [28] A. Ghorbel, W. Hamidouche, and L. Morin, “NeRV++: An enhanced implicit neural video representation,” in *Proc. IEEE Int. Conf. Vis. Commun. and Image Process. (VCIP 2024)*. IEEE, 2024, pp. 1–5. [Online]. Available: <https://doi.org/10.1109/VCIP63160.2024.10849795>
 - [29] X. Zhang, R. Yang, D. He, X. Ge, T. Xu, Y. Wang, H. Qin, and J. Zhang, “Boosting neural representations for videos with a conditional decoder,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR 2024)*, 2024, pp. 2556–2566. [Online]. Available: <https://doi.org/10.1109/CVPR52733.2024.00247>
 - [30] G. Zhang, L. Tang, and X. Zhang, “VQ-NeRV: Vector quantization neural representation for video compression,” in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS 2024)*, 2024, pp. 1–5. [Online]. Available: <https://doi.org/10.1109/ISCAS58744.2024.10558613>
 - [31] J. E. Saethre, R. Azevedo, and C. Schroers, “Combining frame and GOP embeddings for neural video representation,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR 2024)*, 2024, pp. 9253–9263. [Online]. Available: <https://doi.org/10.1109/CVPR52733.2024.00884>
 - [32] H. M. Kwan, G. Gao, F. Zhang, A. Gower, and D. Bull, “NVRC: Neural video representation compression,” in *Adv. Neural Inf. Process. Syst. 37 (NeurIPS 2024)*, 2024, pp. 132 440–132 462.

- [33] J. Wang, Y. Liu, S. Zhu, and C. Feng, “Temporal enhanced hybrid neural representation for video compression,” in *Proc. Picture Coding Symp. (PCS 2024)*, 2024, pp. 1–5. [Online]. Available: <https://doi.org/10.1109/PCS60826.2024.10566352>
- [34] C. Wu, G. Quan, G. He, X.-Q. Lai, Y. Li, W. Yu, X. Lin, and C. Yang, “QS-NeRV: Real-time quality-scalable decoding with neural representation for videos,” in *Proc. 32nd ACM Int. Conf. Multimedia (MM 2024)*, 2024, pp. 2584–2592. [Online]. Available: <https://doi.org/10.1145/3664647.3680586>
- [35] J. Kim, J. Lee, and J.-W. Kang, “SNeRV: Spectra-preserving neural representation for video,” in *Proc. 18th Eur. Conf. Comput. Vis. (ECCV 2024)*, 2024, pp. 332–348. [Online]. Available: https://doi.org/10.1007/978-3-031-73001-6_19
- [36] J. Zhao, X. J. Li, and P. H. J. Chong, “HFS-HNeRV: High-frequency spectrum hybrid neural representation for videos,” in *Proc. 6th ACM Int. Conf. Multimedia Asia (MMAsia 2024)*, 2024. [Online]. Available: <https://doi.org/10.1145/3696409.3700250>
- [37] G. Gao, H. M. Kwan, F. Z. 0017, and D. B. 0001, “PNVC: Towards practical inr-based video compression,” in *AAAI 2025*, 2025, pp. 3068–3076. [Online]. Available: <https://doi.org/10.1609/aaai.v39i3.32315>
- [38] Y. Strümler, J. Postels, R. Yang, L. V. Gool, and F. Tombari, “Implicit neural representations for image compression,” in *Proc. 17th Eur. Conf. Comput. Vis. (ECCV 2022)*, 2022, pp. 74–91. [Online]. Available: https://doi.org/10.1007/978-3-031-19809-0_5
- [39] J. M. Boyce, Y. Ye, J. Chen, and A. K. Ramasubramanian, “Overview of SHVC: Scalable extensions of the high efficiency video coding standard,” *IEEE Trans. Circuits Syst. Video Techn.*, vol. 26, no. 1, pp. 20–34, 2016. [Online]. Available: <https://doi.org/10.1109/TCSVT.2015.2461951>
- [40] M. Tancik, P. P. Srinivasan, B. Mildenhall, S. Fridovich-Keil, N. Raghavan, U. Singhal, R. Ramamoorthi, J. T. Barron, and R. Ng, “Fourier features let networks learn high frequency functions in low dimensional domains,” in *Adv. Neural Inf. Process. Syst. 33 (NeurIPS 2020)*, 2020.
- [41] N. Rahaman, A. Baratin, D. Arpit, F. Dräxler, M. Lin, F. A. Hamprecht, Y. Bengio, and A. C. Courville, “On the spectral bias of neural networks,” in *Proc. 36th Int. Conf. Machine Learn., (ICML 2019)*, vol. 97, 2019, pp. 5301–5310.
- [42] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, “NeRF: Representing scenes as neural radiance fields for view synthesis,” in *Proc. 16th Eur. Conf. Comput. Vis. (ECCV 2022)*, vol. 12346, 2020, pp. 405–421. [Online]. Available: https://doi.org/10.1007/978-3-030-58452-8_24
- [43] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional networks for biomedical image segmentation,” in *Med. Image Comput. Comput.-Assisted Intervention (MICCAI 2015)*, 2015, pp. 234–241. [Online]. Available: http://dx.doi.org/10.1007/978-3-319-24574-4_28
- [44] W. Shi, J. Caballero, F. Huszar, J. Totz, A. P. Aitken, R. Bishop, D. Rueckert, , and Z. Wang, “Real-time single image and video super-resolution using an efficient sub-pixel convolutional neural network,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit., (CVPR 2016)*, 2016. [Online]. Available:

- <http://doi.ieeecomputersociety.org/10.1109/CVPR.2016.207>
- [45] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, “MobileNets: Efficient convolutional neural networks for mobile vis. applications,” *arXiv:1704.04861*, 2017. [Online]. Available: <https://doi.org/10.48550/arXiv.1704.04861>
 - [46] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, “A ConvNet for the 2020s,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR 2022)*, 2022, pp. 11 966–11 976.
 - [47] S. Zagoruyko and N. Komodakis, “DiracNets: Training very deep neural networks without skip-connections,” *arXiv preprint arXiv:2103.03123*, 2017. [Online]. Available: <https://doi.org/10.48550/arXiv.1706.00388>
 - [48] J. Gou, B. Yu, S. J. Maybank, and D. Tao, “Knowledge distillation: A survey,” *Int. J. Comput. Vis.*, vol. 129, no. 6, pp. 1789–1819, 2021. [Online]. Available: <https://doi.org/10.1007/s11263-021-01453-z>
 - [49] G.-H. Wang, J. Li, B. Li, and Y. Lu, “EVC: Towards real-time neural image compression with mask decay,” in *Proc. Int. Conf. Learn. Representations (ICLR 2023)*, 2023.
 - [50] T. Peng, G. Gao, H. Sun, F. Zhang, and D. R. Bull, “Accelerating learnt video codecs with gradient decay and layer-wise distillation,” in *Picture Coding Symp. (PCS 2024)*, 2024, pp. 1–5. [Online]. Available: <https://doi.org/10.1109/PCS60826.2024.10566283>
 - [51] X. Huang and S. J. Belongie, “Arbitrary style transfer in real-time with adaptive instance normalization,” in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV 2017)*, 2017, pp. 1510–1519. [Online]. Available: <https://doi.org/10.1109/ICCV.2017.167>
 - [52] N. Ballas, L. Yao, C. J. Pal, and A. C. Courville, “Delving deeper into convolutional networks for learning video representations,” in *Int. Conf. on Learn. Representations (ICLR 2016)*, 2016.
 - [53] H. Li, A. Kadav, I. Durdanovic, H. Samet, and H. P. Graf, “Pruning filters for efficient convnets,” in *Proc. 5th Int. Conf. Learn. Representations (ICLR 2017)*, 2017.
 - [54] H. Zhou, J. M. Alvarez, and F. Porikli, “Less is more: Towards compact CNNs,” in *Proc. 14th Eur. Conf. Comput. Vis. (ECCV 2016)*, vol. 9908, 2016, pp. 662–677. [Online]. Available: http://dx.doi.org/10.1007/978-3-319-46493-0_40
 - [55] W. Wen, C. Wu, Y. Wang, Y. Chen, and H. Li, “Learning structured sparsity in deep neural networks,” in *Adv. Neural Inf. Process. Syst. 29 (NeurIPS 2016)*, 2016, pp. 2074–2082.
 - [56] S. Girish, K. Gupta, S. Singh, and A. Shrivastava, “LilNetX: Lightweight networks with extreme model compression and structured sparsification,” in *Proc. Int. Conf. Learn. Representations (ICLR 2023)*, Kigali, Rwanda, May 2023.
 - [57] S. Yang and Y. Mao, “Vector quantization of deep convolutional neural networks with learned codebook,” in *Proc. 17th Can. Workshop Inf. Theory (CWIT 2022)*, 2022, pp. 39–44.
 - [58] R. Banner, I. Hubara, E. Hoffer, and D. Soudry, “Scalable methods for 8-bit training of neural networks,” in *Adv. Neural Inf. Process. Syst. 31 (NeurIPS 2018)*, 2018, pp. 5151–5159.

- [59] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. G. Howard, H. Adam, and D. Kalenichenko, “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR 2018)*, 2018, pp. 2704–2713.
- [60] Y. Bengio, N. Léonard, and A. Courville, “Estimating or propagating gradients through stochastic neurons for conditional computation,” *arXiv preprint arXiv:1308.3432*, 2013. [Online]. Available: <https://doi.org/10.48550/arXiv.1308.3432>
- [61] P. Stock, A. Fan, B. Graham, E. Grave, R. Gribonval, H. Jégou, and A. Joulin, “Training with quantization noise for extreme model compression,” in *9th Int. Conf. on Learn. Representations (ICLR 2021)*. OpenReview.net, 2021.
- [62] J. Ballé, V. Laparra, and E. P. Simoncelli, “End-to-end optimized image compression,” in *Int. Conf. on Learn. Representations (ICLR 2017)*, 2017.
- [63] J. Ballé, D. Minnen, S. Singh, S. J. Hwang, and N. Johnston, “Variational image compression with a scale hyperprior,” in *Int. Conf. on Learn. Representations (ICLR 2018)*, 2018.
- [64] C. E. Shannon, “A mathematical theory of communication,” *Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, 1948. [Online]. Available: <https://doi.org/10.1002/j.1538-7305.1948.tb01338.x>
- [65] D. Oktay, J. Ballé, S. Singh, and A. Shrivastava, “Scalable model compression by entropy penalized reparameterization,” in *Proc. 8th Int. Conf. Learn. Represent. (ICLR 2020)*, 2020.
- [66] I. Sodagar, “The MPEG-DASH standard for multimedia streaming over the internet,” *IEEE MultiMedia*, vol. 18, no. 4, pp. 62–67, 2011. [Online]. Available: <http://doi.ieeecomputersociety.org/10.1109/MMUL.2011.71>
- [67] K. Shi, X. Zhou, and S. Gu, “Improved implicit neural representation with fourier reparameterized training,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR 2024)*, 2024, pp. 25 985–25 994. [Online]. Available: <https://doi.org/10.1109/CVPR52733.2024.02455>
- [68] Y. Chen and X. Wang, “Transformers as meta-learners for implicit neural representations,” in *Proc. 17th Eur. Conf. Comput. Vis. (ECCV 2022)*, vol. 13677, 2022, pp. 170–187. [Online]. Available: https://doi.org/10.1007/978-3-031-19790-1_11
- [69] C. Kim, D. Lee, S. Kim, M. Cho, and W.-S. Han, “Generalizable implicit neural representations via instance pattern composers,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR 2023)*, 2023, pp. 11 808–11 817. [Online]. Available: <https://doi.org/10.1109/CVPR52729.2023.01136>
- [70] H. Chen, S. Xie, S.-N. Lim, and A. Shrivastava, “Fast encoding and decoding for implicit video representation,” in *Proc. 18th Eur. Conf. Comput. Vis. (ECCV 2024)*, 2024, pp. 402–418. [Online]. Available: https://doi.org/10.1007/978-3-031-72933-1_23
- [71] P. Micikevicius, S. Narang, J. Alben, G. F. Diamos, E. Elsen, D. García, B. Ginsburg, M. Houston, O. Kuchaiev, G. Venkatesh, and H. Wu, “Mixed precision training,” in *Proc. 6th Int. Conf. Learn. Represent. (ICLR 2018)*, 2018.